Outlier detection learns normal, then scores how far points stray

Unsupervised: no fraud or anomaly labels are needed

LEVEL 1

Domain-based EDA

- Plots, business rules
- Impossible values

LEVEL 2

Univariate rules

- Tukey fences (IQR)
- z-score thresholds

LEVEL 3 · THIS DECK

Multivariate models

- OCSVM · IF · LOF
- MCD · Autoencoder

01 · boundary

OCSVM, SVDD

02 · isolation

Isolation Forest

03 · local density

LOF

04 · rebuild

Autoencoder

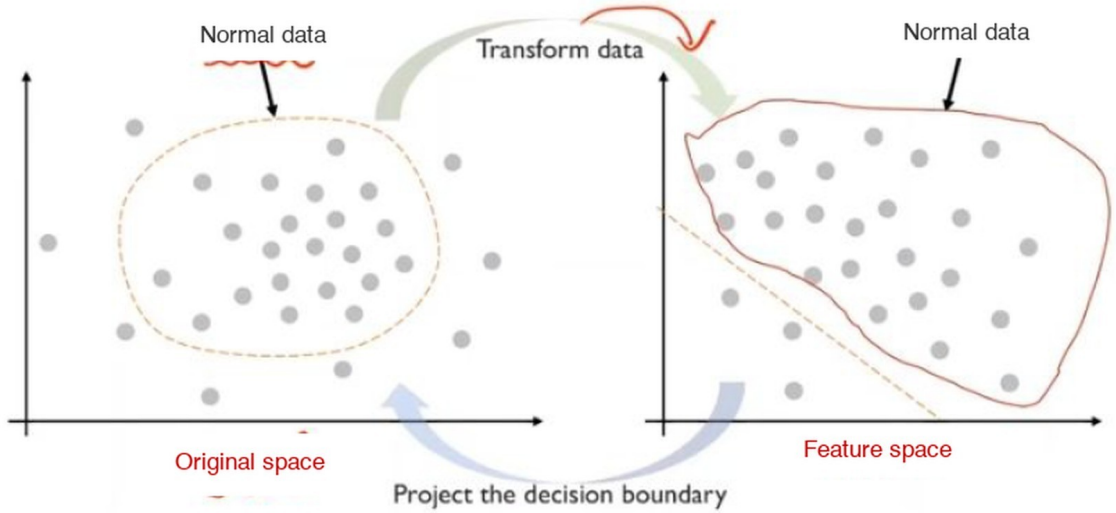
One variable at a time misses points that are odd only in combination

One-Class SVM pushes normal data away from the origin

An SVM trained on one class only: it learns the normal region, not the outliers

Three steps

1. Map the data to a feature space with ϕ
2. Split normal data from the origin with a plane
3. Project the boundary back

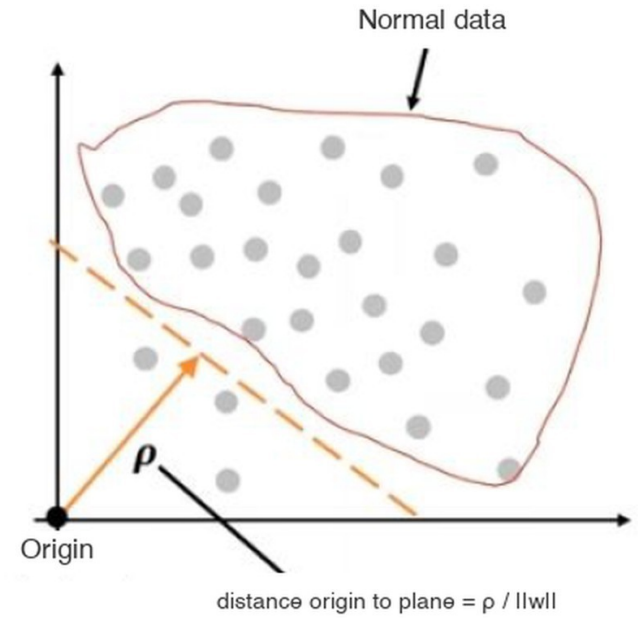


A straight boundary in feature space becomes a curve around normal data

It trains on normal transactions alone, with no fraud labels

The hyperplane sits as far from the origin as possible

Schölkopf et al. (2001): maximise the margin between the origin and the plane



Objective and constraints

$$\min_{w,\xi,\rho} \ \tfrac{1}{2}\|w\|^2 + \tfrac{1}{\nu n}\sum_{i=1}^{n}\xi_i - \rho$$
$$\text{s.t.} \ \ w \cdot \phi(x_i) \geq \rho - \xi_i, \quad \xi_i \geq 0$$

Decision · +1 normal, −1 outlier

$$f(x) = \operatorname{sign}\big(w \cdot \phi(x) - \rho\big), \qquad \text{margin} = \frac{\rho}{\|w\|}$$

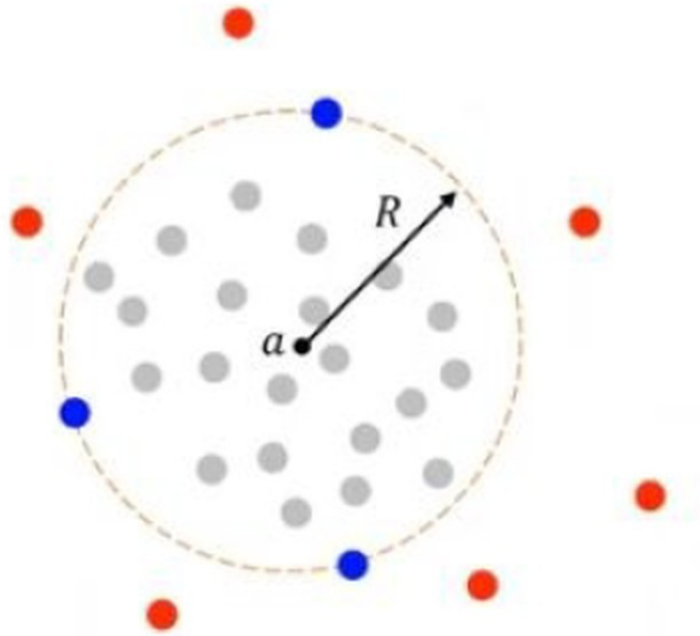
■	$\frac{1}{2}\|w\|^2$	Keep w small: wider margin, smoother boundary
■	$-\rho$	Make ρ large: push the plane from the origin
■	$\frac{1}{\nu n}\sum_i \xi_i$	**Penalty for crossing points; ν caps the share**

A small w and a large ρ both widen the margin, as in an ordinary SVM

The few points allowed to cross (slack ξ) are the outliers

SVDD wraps normal data in the smallest hypersphere

Support Vector Data Description: centre a, radius R, slack ξ for points outside



Objective · shrink the sphere, penalise distance outside

$$\min_{R,a,\xi} \; R^2 + C\sum_{i=1}^{n}\xi_i$$
$$\text{s.t.} \;\; \|\phi(x_i) - a\|^2 \le R^2 + \xi_i, \quad \xi_i \ge 0$$

The dual multiplier α tells where each observation sits

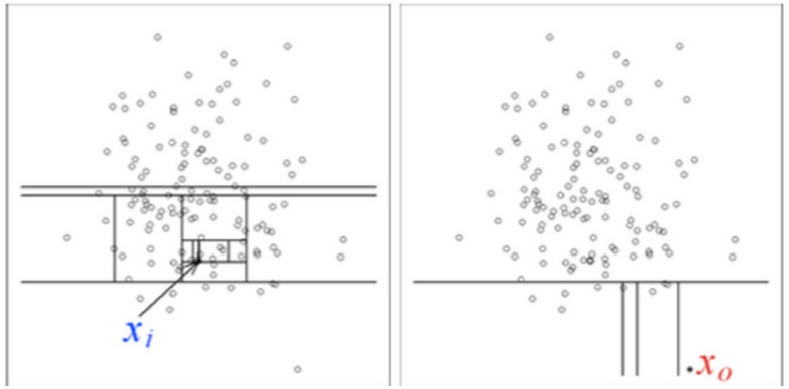
■	$\alpha_i = 0$	Inside the sphere — normal
■	$0 < \alpha_i < C \;\Rightarrow\; \xi_i = 0$	On the sphere — support vector
■	$\alpha_i = C \;\Rightarrow\; \xi_i > 0$	**Outside — support vector, outlier**

Large C keeps almost every point inside; small C gives a tighter sphere and more outliers

With an RBF kernel it matches the One-Class SVM: minimising R = maximising ρ

Isolation Forest flags points that random splits isolate quickly

Pick a feature at random, cut uniformly in [min, max], grow each iTree; no labels needed



Left: crowded x_i needs many cuts · Right: lone x_o needs a few

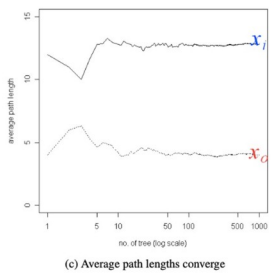
s ≈ 1	much shorter path — outlier
s ≈ 0.5	about average — normal

Anomaly score · h(x): path length

$$s(x, n) = 2^{-E[h(x)]/c(n)}$$

c(n) · average path length

$$c(n) = 2H(n-1) - \frac{2(n-1)}{n} \approx 2\ln n - 1$$



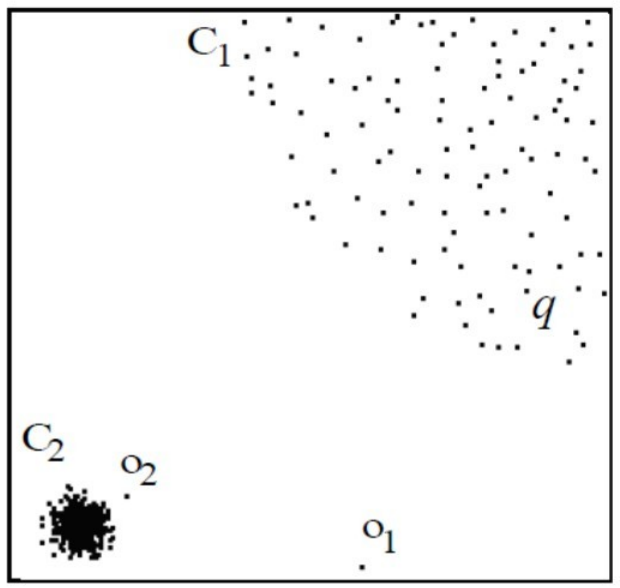
(c) Average path lengths converge

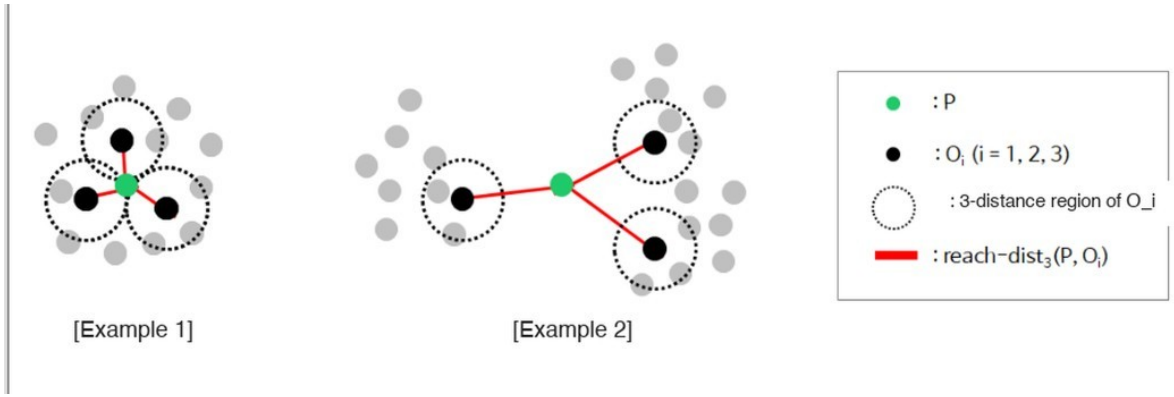
With 256 points c ≈ 10: isolated in 4 splits gives s ≈ 0.76, in 10 splits s ≈ 0.5

Path lengths settle after ~100 trees (normal 12–13, outlier ~4); fast on large data

LOF judges density relative to the neighbours, not absolute distance

Breunig et al. (2000): sparse cluster C1, dense cluster C2, candidate outliers O1 and O2





Reach distance · ≥ o's radius

$$\text{reach}_k(p, o) = \max\{d_k(o),\ d(p, o)\}$$

Local density (lrd) · 1 / mean reach

$$\text{lrd}_k(p) = \frac{|N_k(p)|}{\sum_{o \in N_k(p)} \text{reach}_k(p, o)}$$

Long reach distances from the green p to its neighbours (Example 2) mean low density

A global distance cutoff catches O1 but misses O2 next to the dense C2

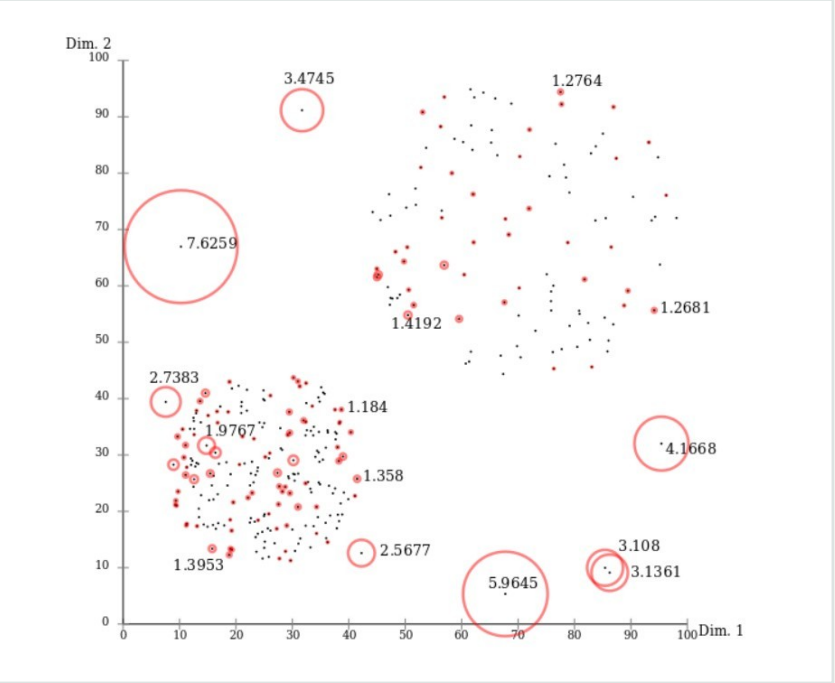
LOF divides the neighbours' density by the point's own

The sparser p and the denser its neighbours, the larger the LOF · numbers in the chart are LOF values

LOF · neighbours' mean density over p's density

$$\mathrm{LOF}_k(p) = \frac{1}{|N_k(p)|} \sum_{o \in N_k(p)} \frac{\mathrm{lrd}_k(o)}{\mathrm{lrd}_k(p)}$$

LOF < 1	Denser than its neighbours — normal
LOF ≈ 1	Similar to its neighbours — normal
LOF ≫ 1	**Sparser than its neighbours — outlier**

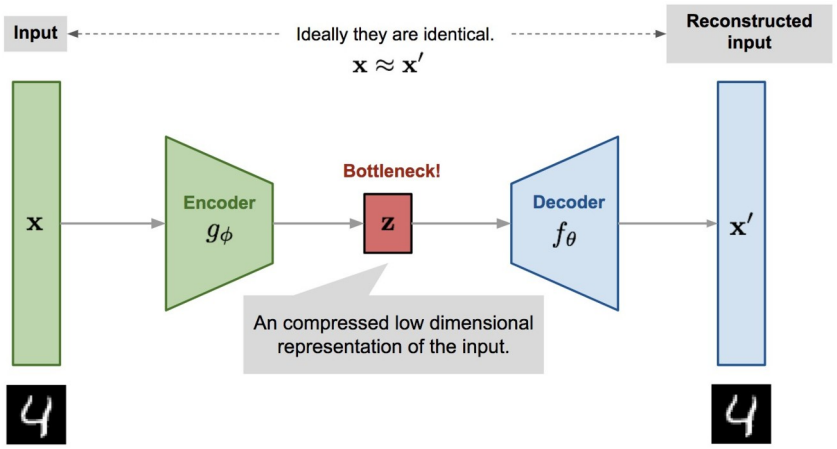


Isolated points score about 2.5 to 7.6; clustered points stay near 1

sklearn's negative_outlier_factor_ is −LOF: more negative = more outlying

An autoencoder trained on normal data cannot rebuild an outlier

Encoder g compresses x into a bottleneck z; decoder f rebuilds x′ ≈ x

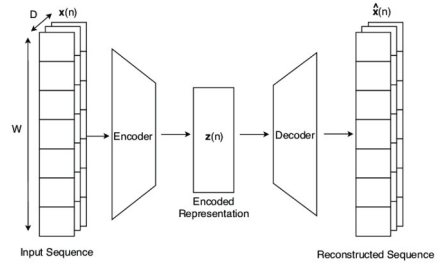


Reconstruction error and cutoff τ

$$e(x) = \|x - f_\theta\big(g_\phi(x)\big)\|^2 > \tau \;\Rightarrow\; \text{outlier}$$

Five steps

1 Split normal vs abnormal
2 Train on normal data only
3 Normal data rebuilds well
4 Outliers rebuild badly
5 Error above τ = outlier



An LSTM autoencoder (lower right) applies the same idea to windows of a time series

It never saw outliers in training, so flag data whose error exceeds τ

Each family defines 'normal' in a different way

Pick the one whose idea of normal matches your data

OCSVM · SVDD

A boundary

Flag	outside
Tune	ν · C

Isolation Forest

Easy to isolate

Flag	$s \approx 1$
Tune	trees

LOF

Locally sparse

Flag	$LOF \gg 1$
Tune	k

Autoencoder

Hard to rebuild

Flag	error > τ
Tune	τ

In scikit-learn, predict returns −1 for an outlier and +1 for a normal point

No labels needed, but always check flagged points with domain sense

※ Code: OneClassSVM · IsolationForest · LocalOutlierFactor (sklearn); LSTM autoencoder example at github.com/cran2367